

Constraint Programming

Zdeněk Hanzálek
zdenek.hanzalek@cvut.cz

CTU in Prague

May 19, 2026

- 1 Inspiration - Sudoku
- 2 Constraint Satisfaction Problem (CSP)
 - How it Works - Propagation and Search
- 3 Constraint Propagation
 - Arc Consistency
 - AC-3 Algorithm
 - Path Consistency
 - Generalized Arc Consistency
 - Bounds Consistency
- 4 Backtracking Search
 - Branching Strategies
- 5 CSP Methodology and Algorithms
- 6 Restarts and Nogoods
- 7 Global Constraints
 - Alldifferent
- 8 Back to Sudoku

What is Constraint Programming?

What is Constraint Programming?

- Sudoku is Constraint Programming

Motivation - Sudoku

			2		5			
	9					7	3	
		2			9		6	
2						4		9
				7				
6		9						1
	8		4			1		
	6	3					8	
			6		8			

Assign digits to blank fields such that:
digits distinct per row, column, block

Sudoku

			2		5			
	9					7	3	
		2			9		6	
2						4		9
				7				
6		9						1
	8		4			1		
	6	3					8	
			6		8			

Assign digits to blank fields such that:
digits distinct per **row**, column, block

Sudoku

			2		5			
	9					7	3	
		2			9		6	
2						4		9
				7				
6		9						1
	8		4			1		
	6	3					8	
			6		8			

Assign digits to blank fields such that:
digits distinct per row, **column**, block

Sudoku

			2		5			
	9					7	3	
		2			9		6	
2						4		9
				7				
6		9						1
	8		4			1		
	6	3					8	
			6		8			

Assign digits to blank fields such that:
digits distinct per row, column, **block**

Sudoku - Propagation in the Lower Left Block

	8	
	6	3

No blank field in the block can have a value of 3,6,8.

A propagation is also called a filtering or pruning.

Sudoku - Propagation in the Lower Left Block

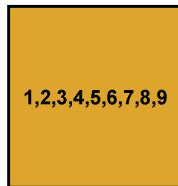
1,2,4,5,7,9	8	1,2,4,5,7,9
1,2,4,5,7,9	6	3
1,2,4,5,7,9	1,2,4,5,7,9	1,2,4,5,7,9

No blank field in the block can have a value of 3,6,8
- propagate to all blank fields

Use the same propagation for rows and columns

Sudoku - Propagation in One Field

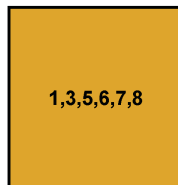
			2		5			
	9					7	3	
		2			9		6	
2						4		9
				7				
6		9						1
	8		4			1		
	6	3					8	
			6		8			



Prune digits from the fields such that:
digits distinct per row, column, block

Sudoku - Propagation in One Field

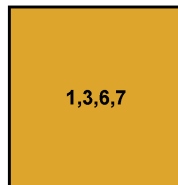
			2		5			
	9					7	3	
		2			9		6	
2						4		9
				7				
6		9						1
	8		4			1		
	6	3					8	
			6		8			



Prune digits from the fields such that:
digits distinct per **row**, column, block

Sudoku - Propagation in One Field

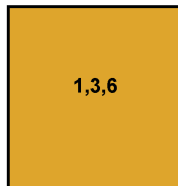
			2		5			
	9					7	3	
		2			9		6	
2						4		9
				7				
6		9						1
	8		4			1		
	6	3					8	
			6		8			



Prune digits from the fields such that:
digits distinct per row, **column**, block

Sudoku - Propagation in One Field

			2		5			
	9					7	3	
		2			9		6	
2						4		9
				7				
6		9						1
	8		4			1		
	6	3					8	
			6		8			



Prune digits from the fields such that:
digits distinct per row, column, **block**

Sudoku - Iterated Propagation

			2		5			
	9					7	3	
		2			9		6	
2						4		9
				7				
6		9						1
	8		4			1		
	6	3					8	
			6		8			

- **Iterate propagation** for rows, columns and blocks
 - When to stop?
 - What if more assignments exist?
 - What if no assignment exists?

Sudoku is Constraint Programming

			2		5			
	9					7	3	
		2			9		6	
2						4		9
				7				
6		9						1
	8		4			1		
	6	3					8	
			6		8			

Sudoku:

- **Variables** - fields
 - assign values - digits
 - maintain **domain** of variable - set of possible values
- **Constraints** - numbers in row, column and box must vary
 - relations among variables
 - disable certain combinations of values

Constraint programming is **declarative programming**:

- **Model**: variables, domains, constraints
- **Solver**: propagation, search

- 1 Inspiration - Sudoku
- 2 **Constraint Satisfaction Problem (CSP)**
 - How it Works - Propagation and Search
- 3 Constraint Propagation
 - Arc Consistency
 - AC-3 Algorithm
 - Path Consistency
 - Generalized Arc Consistency
 - Bounds Consistency
- 4 Backtracking Search
 - Branching Strategies
- 5 CSP Methodology and Algorithms
- 6 Restarts and Nogoods
- 7 Global Constraints
 - Alldifferent
- 8 Back to Sudoku

Constraint Satisfaction Problem - Formal Description

Constraint Satisfaction Problem (CSP) is defined by the triplet (X, D, C) , where:

- $X = \{x_1, \dots, x_n\}$ is a finite set of variables
- $D = \{D_1, \dots, D_n\}$ is a finite set of domains of variables
- $C = \{C_1, \dots, C_t\}$ is a finite set of constraints.

Domain $D_i = \{v_1, \dots, v_k\}$ is a **finite** set of all possible values of x_i .

Constraint C_i is a couple (S_i, R_i) where $S_i \subseteq X$ and R_i is a **relation** over the set of variables S_i . For $S_i = \{x_{i_1}, \dots, x_{i_r}\}$ is $R_i \subseteq D_{i_1} \times \dots \times D_{i_r}$.

CSP is an NP-complete problem.

- **Solution** to Constraint Satisfaction Problem (**CSP**) is the complete **assignment of values** from the domains to the variables such that **all constraints are satisfied**
 - it is a decision problem.
- Constraint Satisfaction Optimization Problem (**CSOP**) is defined by $(X, D, C, f(X))$ where $f(X)$ is the objective function. The search is not finished, when the first acceptable solution was found, but it is finished when the **optimal solution** was found (using branch&bound method for example).
- Constraint Solving is defined by (X, D, C) where D_i is defined on $\mathbb{R}$ (e.g. the solution of the set of linear equations-inequalities).
- Constraint Programming **CP** includes Constraint Satisfaction and Constraint Solving.

How it Works - Propagation and Search

Example: $x \in \{3, 4, 5\}$, $y \in \{3, 4, 5\}$, $x \geq y$, $y > 3$

How it Works - Propagation and Search

Example: $x \in \{3, 4, 5\}$, $y \in \{3, 4, 5\}$, $x \geq y$, $y > 3$

① propagate $y > 3$: $x \in \{3, 4, 5\}$, $y \in \{4, 5\}$

How it Works - Propagation and Search

Example: $x \in \{3, 4, 5\}$, $y \in \{3, 4, 5\}$, $x \geq y$, $y > 3$

- ➊ propagate $y > 3$: $x \in \{3, 4, 5\}$, $y \in \{4, 5\}$
- ➋ propagate $x \geq y$: $x \in \{4, 5\}$, $y \in \{4, 5\}$

How it Works - Propagation and Search

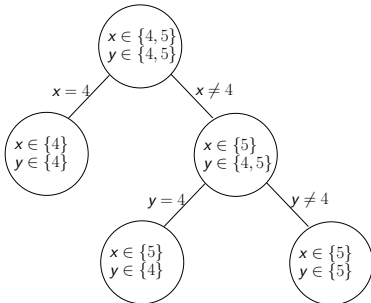
Example: $x \in \{3, 4, 5\}$, $y \in \{3, 4, 5\}$, $x \geq y$, $y > 3$

- ➊ propagate $y > 3$: $x \in \{3, 4, 5\}$, $y \in \{4, 5\}$
- ➋ propagate $x \geq y$: $x \in \{4, 5\}$, $y \in \{4, 5\}$
- ➌ propagation alone is not enough
 - the product of the domains (including infeasible $x = 4$, $y = 5$) is a superset of the solution
 - the search helps - we create subproblems

How it Works - Propagation and Search

Example: $x \in \{3, 4, 5\}$, $y \in \{3, 4, 5\}$, $x \geq y$, $y > 3$

- ❶ propagate $y > 3$: $x \in \{3, 4, 5\}$, $y \in \{4, 5\}$
- ❷ propagate $x \geq y$: $x \in \{4, 5\}$, $y \in \{4, 5\}$
- ❸ propagation alone is not enough
 - the product of the domains (including infeasible $x = 4$, $y = 5$) is a superset of the solution
 - the search helps - we create subproblems
- ❹ in the subproblems we use the propagation again



- The **search** can be driven by **various means** (order of the variables, splitting of domain/domains).
- **Propagation** of the constraints **filters the domains** (see the left branch where $y = 5$ was filtered due to $x \geq y$).

- 1 Inspiration - Sudoku
- 2 Constraint Satisfaction Problem (CSP)
 - How it Works - Propagation and Search
- 3 **Constraint Propagation**
 - Arc Consistency
 - AC-3 Algorithm
 - Path Consistency
 - Generalized Arc Consistency
 - Bounds Consistency
- 4 Backtracking Search
 - Branching Strategies
- 5 CSP Methodology and Algorithms
- 6 Restarts and Nogoods
- 7 Global Constraints
 - Alldifferent
- 8 Back to Sudoku

Arc Consistency (AC)

In AC we consider **binary CSP** - every constraint is a binary relation

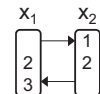
- general (n-ary) CSP can be converted to binary CSP
- binary CSP can be represented by **digraph** G
 - nodes are variables
 - if there is a constraint involving x_i, x_j , then the nodes x_i, x_j are interconnected by oriented arcs (x_i, x_j) and (x_j, x_i)

Arc consistency is an essential method for propagation.

- Arc (x_i, x_j) is **Arc Consistent (AC)** iff for each value $a \in D_i$ there exists $b \in D_j$ such that the assignment $x_i = a, x_j = b$ meets all binary constraints for the variables x_i, x_j .
- A **CSP is Arc Consistent** iff all arcs are arc consistent.

AC is **oriented** - AC of arc (x_i, x_j) does not imply AC of arc (x_j, x_i) .

Three examples with $x_1 \leq x_2$:



no arc is AC



only (x_1, x_2) is AC



both arcs are AC

REVISE Procedure

This procedure deletes every value a from domain D_i , if the value is not consistent with domain D_j .

procedure REVISE

Input: Domain D_i to be revised. Domain D_j . Set of constraints C .

Output: Binary variable *deleted* indicating deletion of some value from D_i . Revised domain D_i .

deleted := 0;

for $a \in D_i$ **do**

if *there is no* $b \in D_j$; $x_i = a, x_j = b$ *satisfies all* x_i, x_j *constraints*
 then

$D_i := D_i \setminus a$;

 // delete a from D_i

deleted := 1;

end

end

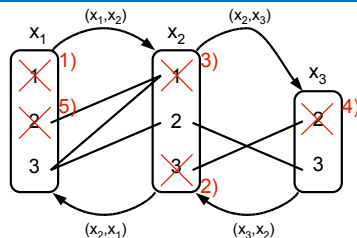
Example: Application of REVISE

CSP with variables and domains

$x_1 \in \{1, 2, 3\}, x_2 \in \{1, 2, 3\}, x_3 \in \{2, 3\}$

and constraints

$x_1 > x_2, x_2 \neq x_3, x_2 + x_3 > 4.$



revised arc	deleted	revised domain	(x_1, x_2)	(x_2, x_1)	(x_2, x_3)	(x_3, x_2)
(x_1, x_2)	1 ¹⁾	$x_1 \in \{2, 3\}$	AC	non-AC	non-AC	AC
(x_2, x_1)	3 ²⁾	$x_2 \in \{1, 2\}$	AC	AC	non-AC	non-AC
(x_2, x_3)	1 ³⁾	$x_2 \in \{2\}$	non-AC	AC	AC	non-AC
(x_3, x_2)	2 ⁴⁾	$x_3 \in \{3\}$	non-AC	AC	AC	AC

After revision, some of the arcs are still **nonconsistent**

- the reason is that some of the domains have been reduced
- continue in the revision until all the arc are consistent (but you have to do it without perpetual AC checking of all arcs - see AC-3 alg.)

revised arc	deleted	revised domain	(x_1, x_2)	(x_2, x_1)	(x_2, x_3)	(x_3, x_2)
(x_1, x_2)	2 ⁵⁾	$x_1 \in \{3\}$	AC	AC	AC	AC

Arc Consistency - AC-3 Algorithm

Maintain a queue of arcs to be revised (the arc is put in the queue only if it's consistency could have been affected by the reduction of the domain).

procedure AC-3

Input: X, D, C and graph G .

Output: Binary variable *fail* indicating no solution in this part of the state space. The set of the revised domains D .

```
fail = 0; Q := E(G);           // initialize Q by arcs of G
while Q ≠ ∅ do
    select and remove arc  $(x_k, x_m)$  from Q;
    (deleted,  $D_k$ ) = REVISE( $D_k, D_m, C$ );
    if deleted then
        if  $D_k = \emptyset$  then fail = 1 and EXIT ;
        else Q := Q ∪ {( $x_i, x_k$ ) such that  $(x_i, x_k) \in E(G)$  and  $i \neq m$ };
    end
end
```

The revision of (x_k, x_m) does not change the arc consistency of (x_m, x_k) .

Example: Iteration of AC-3

CSP with variables and dom. $x_1 \in \{1, 2, 3\}$, $x_2 \in \{1, 2, 3\}$, $x_3 \in \{1, 2, 3\}$
and constraints $x_1 = x_2$, $x_2 + 1 = x_3$.

Queue initialization: $Q = ((x_1, x_2), (x_2, x_1), (x_2, x_3), (x_3, x_2))$

revise (x_1, x_2)

$x_1 \in \{1, 2, 3\}$, $x_2 \in \{1, 2, 3\}$, $x_3 \in \{1, 2, 3\}$

$Q = \{(x_2, x_1), (x_2, x_3), (x_3, x_2)\}$

revise (x_2, x_1)

$x_1 \in \{1, 2, 3\}$, $x_2 \in \{1, 2, 3\}$, $x_3 \in \{1, 2, 3\}$

$Q = \{(x_2, x_3), (x_3, x_2)\}$

revise (x_2, x_3)

$x_1 \in \{1, 2, 3\}$, $x_2 \in \{1, 2\}^1$, $x_3 = \{1, 2, 3\}$

$Q = \{(x_3, x_2), (x_1, x_2)\}$

revise (x_3, x_2)

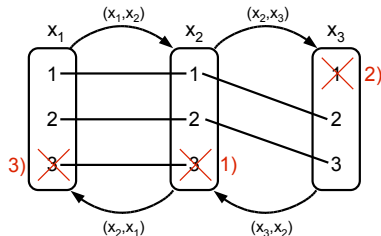
$x_1 \in \{1, 2, 3\}$, $x_2 \in \{1, 2\}$, $x_3 \in \{2, 3\}^2$

$Q = \{(x_1, x_2)\}$

revise (x_1, x_2)

$x_1 \in \{1, 2\}^3$, $x_2 \in \{1, 2\}$, $x_3 \in \{2, 3\}$

$Q = \emptyset$

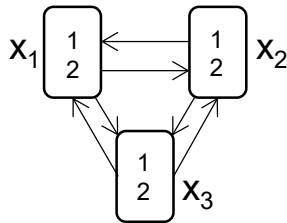


Arc Consistency is not Complete

Example:

CSP with variables and domains $x_1 \in \{1, 2\}$, $x_2 \in \{1, 2\}$, $x_3 \in \{1, 2\}$ and constraints $x_1 \neq x_2$, $x_1 \neq x_3$, $x_2 \neq x_3$.

It is **not feasible**, but it is AC.



This example is not Path Consistent: any assignment of x_1 and x_3 satisfying $x_1 \neq x_3$ is not consistent over path x_1, x_2, x_3 .

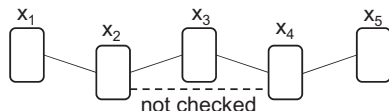
Path Consistency (PC)

In PC we consider more variables with **binary constraints** at once.

- Path $x_1, \dots, x_m$ is **Path Consistent (PC)** iff **for every pair of values** $a \in D_1$ $b \in D_m$ satisfying all the binary constraints on x_1, x_m **there exists an assignment of variables** $x_2, \dots, x_{m-1}$ such that all the binary constraints between the neighbouring variables x_i, x_{i+1} are satisfied.
- A **CSP is Path Consistent** iff all paths are path consistent.

Example continued from previous slide: Path x_1, x_2, x_3 is not PC. For $x_1 = 1, x_3 = 2$, which is consistent w.r.t. $x_1 \neq x_3$ there is no assignment of x_2 such that $x_1 \neq x_2$ and $x_2 \neq x_3$ are satisfied.

But only the constraints between the neighboring variables must be satisfied. For given path $(x_1, \dots, x_5)$:



Path Consistency is not Complete

CSP is PC if and only if all paths of length 2 are PC.

PC implies AC.

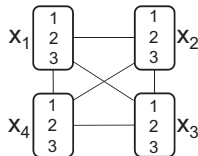
PC is still not a **complete technique**.

Example:

CSP with variables and domains $x_1 \in \{1, 2, 3\}$, $x_2 \in \{1, 2, 3\}$, $x_3 \in \{1, 2, 3\}$, $x_4 \in \{1, 2, 3\}$ and constraints $x_1 \neq x_2, x_1 \neq x_3, x_1 \neq x_4, x_2 \neq x_3, x_2 \neq x_4, x_3 \neq x_4$.

This example is PC but it is **not feasible**.

Part of the check is consistent path (x_1, x_2, x_3, x_4) with values $x_1 = 1, x_2 = 2, x_3 = 1, x_4 = 3$.



PC removes more (or the same) inconsistencies than AC, but the **strength/efficiency ratio of PC** is much worse than for AC.

Generalized Arc Consistency (GAC) - More Variables

In GAC we consider **one constraint with more than 2 variables**.

Value a of variable x_i is **Generalized Arc Consistent (GAC)** with respect to constraint C_j iff there exist values in domains of the other variables of C_j such that together with a they satisfy constraint C_j .

Example: $x_1 \in \{1, 2, 3\}$, $x_2 \in \{1, 2, 3\}$, $x_3 \in \{1, 2, 3\}$ and constraint $x_1 + x_2 \leq x_3$. For x_3 : value 1 is not GAC, but values 2 and 3 are GAC.

Variable x_i is **GAC** with respect to constraint C_j , iff all values from the domain of x_i are GAC with respect to C_j .

Example: $x_1 \in \{1, 2, 3\}$, $x_2 \in \{1, 2, 3\}$, $x_3 \in \{2, 3\}$ and $x_1 + x_2 \leq x_3$. Variable x_3 is GAC, but x_1 and x_2 are not GAC.

Constraint C_j is **GAC**, iff all variables in C_j are GAC.

Ex: $x_1 \in \{1, 2\}$, $x_2 \in \{1, 2\}$, $x_3 \in \{2, 3\}$ constraint $x_1 + x_2 \leq x_3$ is GAC.

A **CSP is GAC** iff all the constraints in CSP are GAC.

Checking GAC is **NP-hard** for some types of constraints.

We consider a **weaker version of GAC**: the GAC property is required only for the **boundary values of the checked variable**.

We assume the **values to be ordered** in their domains.

Each variable domain can then be seen as an **interval represented by two bounds**.

Bounds Consistency and its Versions

Variable x_i is BC with respect to constraint C_j , iff **the lower bound and upper bound values** of the domain of x_i are GAC with respect to C_j .

Consequently: Constraint C_j is BC, iff all variables in C_j are BC. A CSP is BC iff all the constraints in CSP are BC.

Example: $x_1 \in \{1, 2, 3\}$, $x_2 \in \{0, 1\}$, $x_3 \in \{1\}$ constraint $x_1 = 2x_2 + x_3$.
Variable x_1 is BC, but it is not GAC for $x_1 = 2$.

There are faster checks:

Bounds $\mathcal{Z}$ Consistency(BZC) - for the value check we do not consider domains of the other variables of C_j , that may be sparse, but rather **integer intervals** ranging from the smallest to the largest value of the respective domain. BZC is weaker than BC: if x_i is BC, then it is BZC.

Example: $x_1 \in \{1, 2, 3\}$, $x_2 \in \{0, 2\}$, $x_3 \in \{0, 1\}$ constraint $x_1 = 2x_2 + x_3$.
Variable x_1 is BZC, but it is not BC for $x_1 = 3$.

Bounds $\mathcal{R}$ Consistency(BRC) - we consider intervals of real numbers.

- 1 Inspiration - Sudoku
- 2 Constraint Satisfaction Problem (CSP)
 - How it Works - Propagation and Search
- 3 Constraint Propagation
 - Arc Consistency
 - AC-3 Algorithm
 - Path Consistency
 - Generalized Arc Consistency
 - Bounds Consistency
- 4 Backtracking Search
 - Branching Strategies
- 5 CSP Methodology and Algorithms
- 6 Restarts and Nogoods
- 7 Global Constraints
 - Alldifferent
- 8 Back to Sudoku

Backtracking search

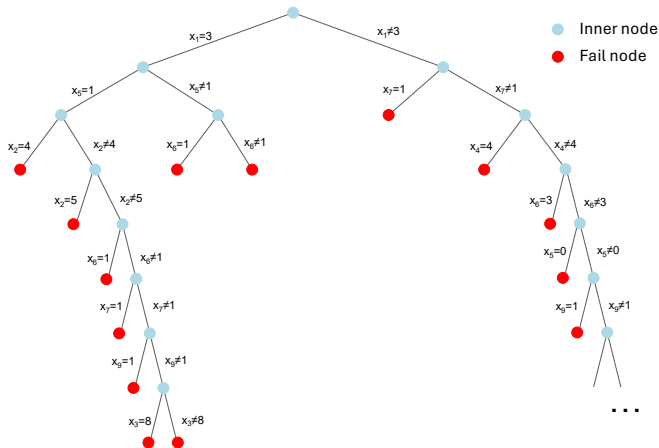
A backtracking search is a **depth-first traversal of a search tree**

- the search tree is **generated** as the search progresses
- the search tree represents **alternative choices** that may have to be examined in order to find a solution
- the **backtrack** occurs when any domain of a variable becomes empty
- the search **stops** when a feasible solution is found or the complete tree was explored without feasible solution.

Branching strategy - Method of extending a node in the search tree

- A node $p = \{b_1, \dots, b_j\}$ in the search tree is a set of **branching constraints**, where $b_i, 1 \leq i \leq j$, is the branching constraint posted at level i in search tree.
- A node p is extended by posting a branching constraint to **ensure completeness**, the constraints posted on all the branches from a node must be **mutually exclusive and exhaustive**.

Example - Search Tree with Binary Choice Branching



Popular Branching Strategies

Example: let x_1 be the variable branched on, and let $D_1 = \{1, \dots, 6\}$

Binary choice (or 2-way branching)

- variable x_1 is instantiated to some value in its domain D_1
- e.g., $x_1 = 1$ is posted along the first branch, $x_1 \neq 1$ along second branch, respectively

Domain splitting

- constraint splits the domain of the variable
- e.g., $x_1 \leq 3$ is posted along the first branch, $x_1 > 3$ along second branch, respectively

Enumeration (or d-way branching)

- variable x_1 is instantiated in turn to each value in its domain
- e.g., $x_1 = 1$ is posted along the first branch, $x_1 = 2$ along second branch, $x_1 = 3$ along third branch, etc.

Posting non-unary branching constraints, branching strategies that are **specific to class of problems**

Example: relative order of tasks in scheduling

- we must schedule a set of tasks $T_1, \dots, T_n$ on a given resource
- let x_i be a variable representing the start time of task T_i
- let p_i be the fixed processing time of task T_i
- idea: serialize the tasks that share a resource
- example: consider two task T_1 and T_2 which share a resource
 - post the constraint $x_1 + p_1 \leq x_2$ along first branch
 - post the constraint $x_2 + p_2 \leq x_1$ along the second branch

- 1 Inspiration - Sudoku
- 2 Constraint Satisfaction Problem (CSP)
 - How it Works - Propagation and Search
- 3 Constraint Propagation
 - Arc Consistency
 - AC-3 Algorithm
 - Path Consistency
 - Generalized Arc Consistency
 - Bounds Consistency
- 4 Backtracking Search
 - Branching Strategies
- 5 CSP Methodology and Algorithms
- 6 Restarts and Nogoods
- 7 Global Constraints
 - Alldifferent
- 8 Back to Sudoku

Model the problem

- define/choose constraint model: variables, domains, constraints

Solve the model while using many **techniques for improving efficiency** of a backtracking search algorithm:

- constraint propagation,
- backtracking search,
- restarts and nogood recording - see in the next slides,
- global constraint - see in the next slides,
- heuristics for variable and value ordering.

These techniques are **not always orthogonal**; combining can give a multiplicative effect and degradation effect.

Effective backtracking algorithms perform **constraint propagation during the search**.

- Typically they perform constraint propagation **at each node** in the search tree.

Backtracking search integrated with constraint propagation has two important **benefits**:

- removing inconsistencies during search can dramatically **prune the search tree** by removing deadends and by simplifying the remaining sub-problem
- some of the most important **variable ordering heuristics make use of the information** gathered by constraint propagation

Some CSP Algorithms

Chronological backtracking

- naive backtracking: performs **no constraint propagation**, only checks a constraint if **all of its variables have been instantiated**; chronologically backtracks

Forward checking

- maintains arc consistency on all constraints with **exactly one uninstantiated variable**; chronologically backtracks
- it is the easiest example of "look ahead strategy"

Maintaining arc consistency

- maintains (generalized) **arc consistency on all constraints**; chronologically backtracks

Maintaining bounds consistency

- maintains **bounds consistency on all constraints**; chronologically backtracks

Example Problem - Constraint Model for 4-queens

Place 4 queens on an 4x4 board so that no pair attacks each other:

- value of x_i is a **row index of queen** in column i
- one queen in column i - given by **one value chosen for variable** x_i
- no pair of queens in the same row - given by 6 inequalities $x_i \neq x_j$
- no pair of queens on the same diagonal - given by 6 ineq. $|x_i - x_j| \neq k$

variables:

x_1, x_2, x_3, x_4

domains:

$\{1, 2, 3, 4\}$

constraints:

$$x_1 \neq x_2 \wedge |x_1 - x_2| \neq 1$$

$$x_1 \neq x_3 \wedge |x_1 - x_3| \neq 2$$

$$x_1 \neq x_4 \wedge |x_1 - x_4| \neq 3$$

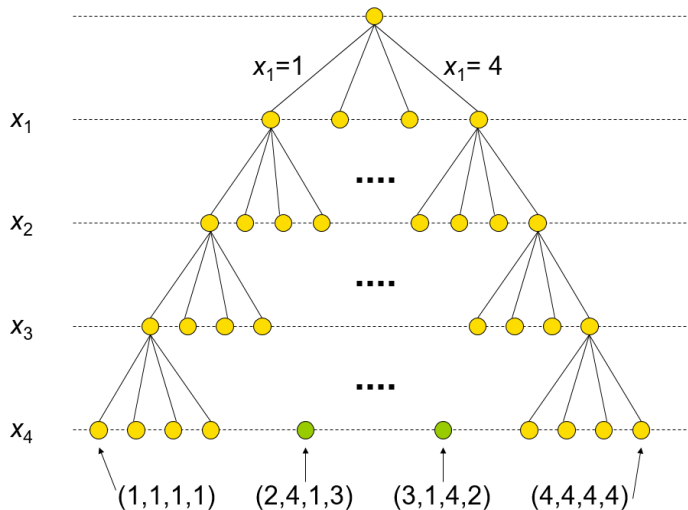
$$x_2 \neq x_3 \wedge |x_2 - x_3| \neq 1$$

$$x_2 \neq x_4 \wedge |x_2 - x_4| \neq 2$$

$$x_3 \neq x_4 \wedge |x_3 - x_4| \neq 1$$

	x_1	x_2	x_3	x_4
1				
2				
3				
4				

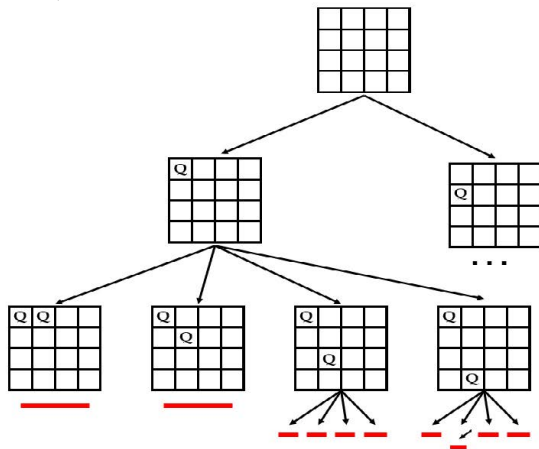
Enumerated Search Tree for 4-queens



Chronological Backtracking - Search Progress

Check all constraint with all instantiated variables

e.g. $x_1 \neq x_2$



Forward Checking

Reduce domains on all constraints with exactly one uninstantiated variable

$$\{x_1 = 1\}$$

constraints:

$$x_1 \neq x_2 \wedge |x_1 - x_2| \neq 1$$

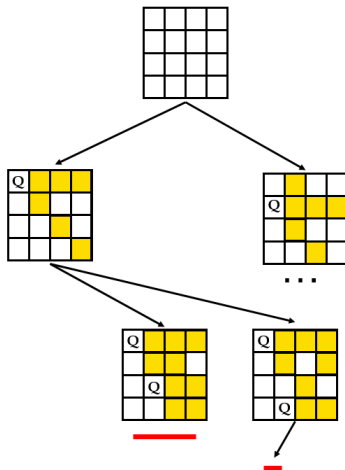
$$x_1 \neq x_3 \wedge |x_1 - x_3| \neq 2$$

$$x_1 \neq x_4 \wedge |x_1 - x_4| \neq 3$$

	x_1	x_2	x_3	x_4
1	Q			
2				
3				
4				

yellow row and yellow diagonals

Forward Checking - Search Progress



on this level:
 D_3 is empty

on the next level:
 $x_3 = 2$ and D_4 is empty

Maintaining Arc Consistency

Check all constraints

- AC-3 algorithm stops since D_3 is empty

$$\{x_1 = 1\}$$

constraints:

$$x_1 \neq x_2 \wedge |x_1 - x_2| \neq 1$$

$$x_1 \neq x_3 \wedge |x_1 - x_3| \neq 2$$

$$x_1 \neq x_4 \wedge |x_1 - x_4| \neq 3$$

$$x_2 \neq x_3 \wedge |x_2 - x_3| \neq 1$$

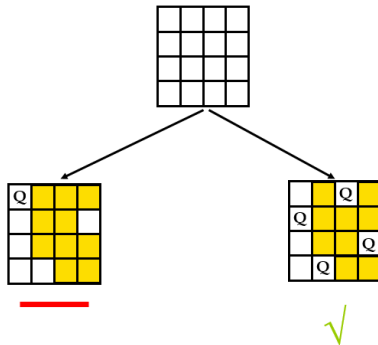
$$x_2 \neq x_4 \wedge |x_2 - x_4| \neq 2$$

$$x_3 \neq x_4 \wedge |x_3 - x_4| \neq 1$$

	x_1	x_2	x_3	x_4
1	Q			
2				
3				
4				

Maintaining Arc Consistency - Result

Feasible solution found in the second vertex.



Example Problem - Simple Equation and Inequality

$x_1 \in \{2, 3\}, x_2 \in \{1, 2, 3\}, x_3 \in \{4, 5\}$
constraints $x_1 > x_2$ and $x_1 + x_2 = x_3$

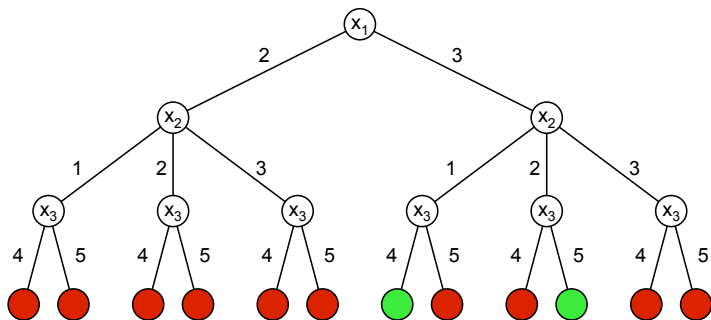
Enumerated Search Tree

green leaf - feasible solution

$$x_1 \in \{2, 3\}, x_2 \in \{1, 2, 3\}, x_3 \in \{4, 5\}$$

$$x_1 > x_2$$

$$x_1 + x_2 = x_3$$



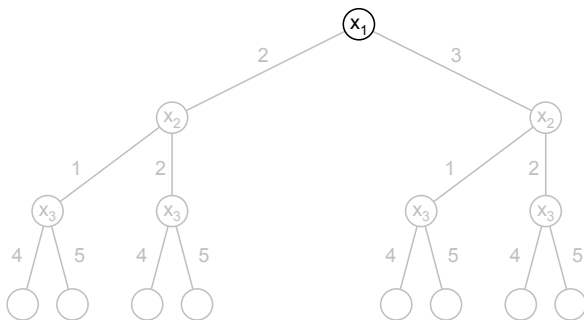
Maintaining Arc Consistency

Initial propagation of constraints:

$$x_1 \in \{2, 3\}, x_2 \in \{1, 2, \textcolor{red}{3}\}, x_3 \in \{4, 5\}$$

$$x_1 > x_2$$

$$x_1 + x_2 = x_3$$



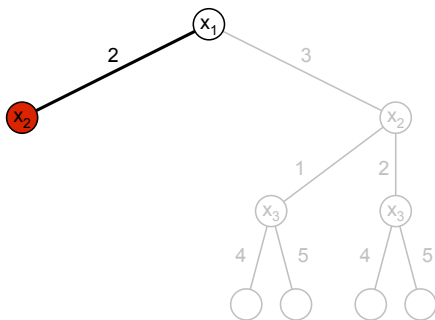
Maintaining Arc Consistency

Choose $x_1 = 2$ and GAC:

$$x_1 \in \{2, 3\}, x_2 \in \{1, \cancel{2}, \cancel{3}\}, x_3 \in \{\cancel{4}, \cancel{5}\}$$

$$x_1 > x_2$$

$$x_1 + x_2 = x_3$$



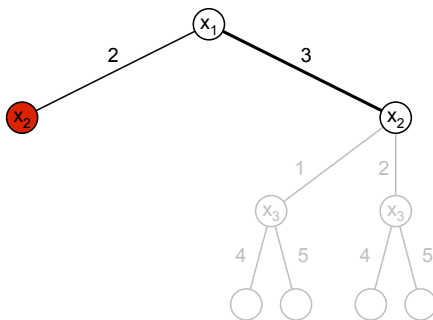
Maintaining Arc Consistency

Choose $x_1 = 3$ and GAC:

$$x_1 \in \{2, \textcircled{3}\}, x_2 \in \{1, 2, \textcolor{red}{\cancel{3}}\}, x_3 \in \{4, 5\}$$

$$x_1 > x_2$$

$$x_1 + x_2 = x_3$$



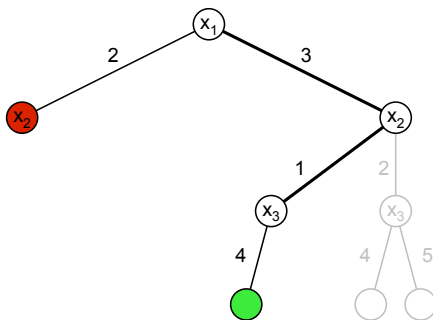
Maintaining Arc Consistency

Choose $x_2 = 1$ and GAC:

$$x_1 \in \{2, \textcircled{3}\}, x_2 \in \{\textcircled{1}, 2, \textcolor{red}{\cancel{3}}\}, x_3 \in \{4, \textcolor{red}{\cancel{5}}\}$$

$$x_1 > x_2$$

$$x_1 + x_2 = x_3$$



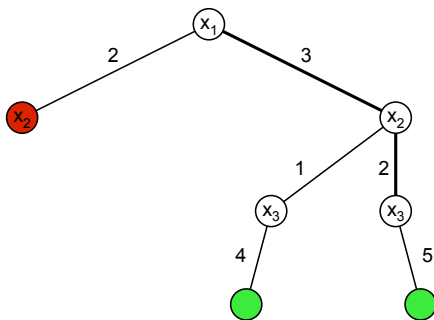
Maintaining Arc Consistency

Choose $x_2 = 2$ and GAC:

$$x_1 \in \{2, \textcircled{3}\}, x_2 \in \{1, \textcircled{2}, \textcolor{red}{3}\}, x_3 \in \{\textcolor{red}{4}, 5\}$$

$$x_1 > x_2$$

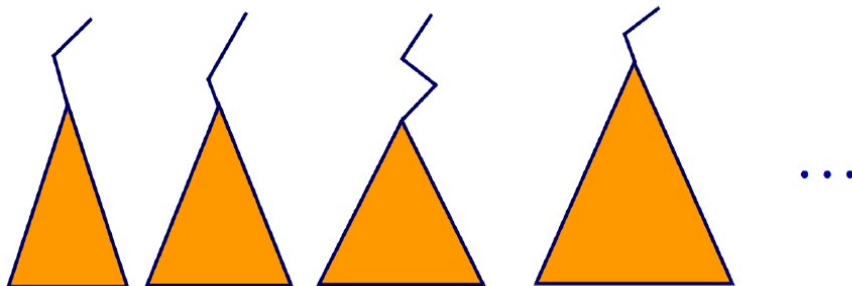
$$x_1 + x_2 = x_3$$



- 1 Inspiration - Sudoku
- 2 Constraint Satisfaction Problem (CSP)
 - How it Works - Propagation and Search
- 3 Constraint Propagation
 - Arc Consistency
 - AC-3 Algorithm
 - Path Consistency
 - Generalized Arc Consistency
 - Bounds Consistency
- 4 Backtracking Search
 - Branching Strategies
- 5 CSP Methodology and Algorithms
- 6 Restarts and Nogoods
- 7 Global Constraints
 - Alldifferent
- 8 Back to Sudoku

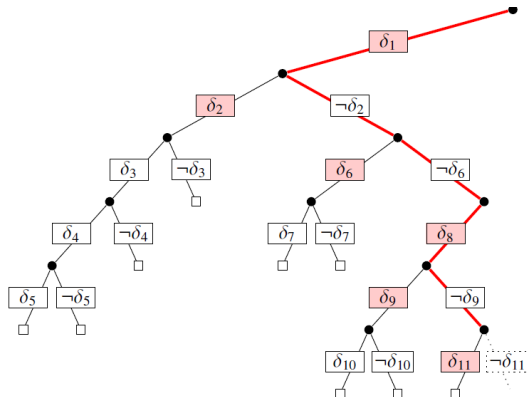
Restarts and Nogoods

Restarting allows to reconsider choicess made at the top of the search tree
- **different order of variable-value choices.**



Without nogood recording, the search after **restart** may **duplicate some work**.

Nogoods



Nogoods:

- ▶ $\neg\delta_1 \vee \neg\delta_2$
- ▶ $\neg\delta_1 \vee \neg\delta_6$
- ▶ $\neg\delta_1 \vee \neg\delta_8 \vee \neg\delta_9$
- ▶ $\neg\delta_1 \vee \neg\delta_8 \vee \neg\delta_{11}$

- 1 Inspiration - Sudoku
- 2 Constraint Satisfaction Problem (CSP)
 - How it Works - Propagation and Search
- 3 Constraint Propagation
 - Arc Consistency
 - AC-3 Algorithm
 - Path Consistency
 - Generalized Arc Consistency
 - Bounds Consistency
- 4 Backtracking Search
 - Branching Strategies
- 5 CSP Methodology and Algorithms
- 6 Restarts and Nogoods
- 7 Global Constraints
 - Alldifferent
- 8 Back to Sudoku

Global constraint

- captures **specific structure** of the problem
- uses this structure for efficient propagation in **specialized propagation algorithm**

Other examples of global constraints:

- scheduling (edge-finder)
- graph algorithms (clique, cycle)
- finite state machine
- bin-packing

Alldifferent Global Constraint

On set $X = \{x_1, \dots, x_n\}$ we apply constraint $x_i \neq x_j \ \forall i \neq j$

- This can be formulated by $(n^2 - n)/2$ inequalities.
- The second option is the global constraint **alldifferent**, which uses a matching algorithm in a bipartite graph, where one side represents the variables and the other side represents the values.

Example: **alldifferent**(x_1, x_2, x_3, x_4)

where the domains are:

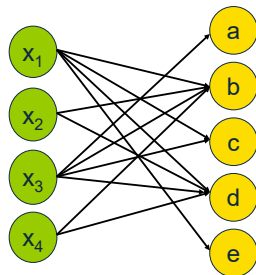
$$D_1 = \{b, c, d, e\}$$

$$D_2 = \{b, d\}$$

$$D_3 = \{a, b, c, d\}$$

$$D_4 = \{b, d\}$$

Solution $x_1 = e, x_2 = b, x_3 = c, x_4 = d$.



- 1 Inspiration - Sudoku
- 2 Constraint Satisfaction Problem (CSP)
 - How it Works - Propagation and Search
- 3 Constraint Propagation
 - Arc Consistency
 - AC-3 Algorithm
 - Path Consistency
 - Generalized Arc Consistency
 - Bounds Consistency
- 4 Backtracking Search
 - Branching Strategies
- 5 CSP Methodology and Algorithms
- 6 Restarts and Nogoods
- 7 Global Constraints
 - Alldifferent
- 8 Back to Sudoku

Sudoku - Variables

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9
x_{10}	x_{11}	x_{12}	x_{13}	x_{14}	...			
x_{19}	x_{20}	x_{21}	...					
x_{28}	...							
x_{37}	...							
x_{46}	...							
x_{55}	...							
x_{64}	...							
x_{73}	...							

Sudoku - Domains and Constraints

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9
x_{10}	x_{11}	x_{12}	x_{13}	x_{14}	...			
x_{19}	x_{20}	x_{21}	...					
x_{28}	...							
x_{37}	...							
x_{46}	...							
x_{55}	...							
x_{64}	...							
x_{73}	...							

$$D_i = \{1, \dots, 9\}, \forall i = 1, \dots, 81$$

$\text{alldifferent}(x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9)$

$\vdots$

$\text{alldifferent}(x_1, x_{10}, x_{19}, x_{28}, x_{37}, x_{46}, x_{55}, x_{64}, x_{73})$

$\vdots$

$\text{alldifferent}(x_1, x_2, x_3, x_{10}, x_{11}, x_{12}, x_{19}, x_{20}, x_{21})$

$\vdots$

$$x_1 = 5, x_2 = 3, x_5 = 7, \dots, x_{81} = 9$$

Example using 3 different CSP algorithms:

- Forward checking
- Maintaining arc consistency using GAC
- Maintaining bounds consistency using BZC

- In both cases we deal with declarative programming
- Performance differs from problem to problem
- CSP allows one to formulate **complex constraints**
(ILP uses inequalities only, CSP uses an arbitrary relation - e.g. a binary relation may be given by a set of compatible tuples)
 - CSP is more flexible, formulation is easier to understand
- it is difficult to represent continuous problems by CSP
 - domains of real variables can be bypassed by using hybrid approaches
- e.g. combination with LP
- CP is new technique, it is more open

Tools for Solving CSP

Proprietary:

- IBM CP Optimizer (C++)
- OptaCP

Open source:

- CP-SAT Solver by Google OR-tools
- Tempo
- ECLiPSe (Prolog)
- Gecode (C++)
- Choco Solver (Java)
- Python constraints
- MiniZinc - high-level constraint modelling

References



Roman Barták.

Programování s omezujícími podmínkami, 2010.



Rina Dechter.

Constraint Processing.

Morgan Kaufmann, 2003.



Vilém Heinz, Petr Vilím, and Zdeněk Hanzálek.

Reinforcement learning for search tree size minimization in constraint programming: New results on scheduling benchmarks.

Computers and Industrial Engineering, 209:111413, 2025.



Christian Schulte.

Constraint programming, 2010.



Peter van Beek.

Introduction to constraint programming.

CPAIOR, 2009.